

Nifty 100 Financial Intelligence Platform

Analyst User Guide

Sprint 6 - API, Clustering & Sign-off

A practical handbook for equity-research analysts covering the Streamlit dashboard, PDF tearsheet generation, and the REST API.

1. Platform Overview

The Nifty 100 Financial Intelligence Platform is a Python-based analytics workspace covering 92 large- and mid-cap Indian equities with up to 14 years of financial history. It combines four layers:

- **ETL pipeline** - ingests 12 Screener.in Excel files, normalises company identifiers and financial-year labels, runs 16 data-quality rules, and loads the data into a SQLite warehouse (`db/nifty100.db`).
- **KPI engine** - computes 60+ financial ratios including ROE, ROCE, D/E, ICR, free cash flow, CAGR, valuation multiples and a 0-100 composite quality score.
- **Analytics** - peer-group percentiles, capital-allocation pattern detection, outlier detection, KMeans clustering into 5 archetypes (High-Quality Compounder, Value Cyclical, Defensive Dividend Payer, Emerging Growth, Distressed / Turnaround).
- **Surfaces** - an 8-screen Streamlit dashboard, a FastAPI REST API (`/api/v1`), and batch PDF tearsheet generation.

The dashboard and API are read-only views over the warehouse. Refresh the data by re-running the ETL scripts from the `scripts/` directory.

2. Setup & Launch

2.1 Pre-requisites

- Python 3.11 or newer (tested on 3.13).
- The dependencies listed in `pyproject.toml` (FastAPI, Streamlit, Pandas, scikit-learn, ReportLab, Plotly, ...).
- Raw data files in `data/raw/` (7 core Screener.in exports) and `data/raw/supporting datasets/` (5 supplementary files).

2.2 Install

```
pip install -r requirements.txt # or install via pip as needed
```

2.3 Run the ETL (first time / refresh)

```
python scripts/day1_ingest.py # import raw data
python scripts/day2_normalise.py # normalise ids/years
python scripts/populate_ratios.py # compute financial ratios
python scripts/day36_clustering.py # KMeans clustering
python scripts/generate_tearsheets.py # 92 per-company PDFs
```

2.4 Launch the dashboard and API

```
# FastAPI on http://127.0.0.1:8000
uvicorn src.api.main:app --port 8000 --host 0.0.0.0

# Streamlit dashboard on http://127.0.0.1:8501
streamlit run src/dashboard/app.py
```

Both servers can run simultaneously without port conflict (the API is on 8000, the dashboard on 8501). The dashboard reads data directly from the SQLite database, not from the API, but the API can be used to power custom scripts or external tools.

3. Dashboard Screens

The Streamlit dashboard has eight screens, selectable from the left-hand sidebar.

3.1 Home

Landing page with headline statistics (92 companies, 11 sectors, date range) and an overview of the five cluster archetypes with company counts.

3.2 Company Profile

Select a ticker from the drop-down (or type to search). The page renders:

- Company name, sector, market-cap category, website and NSE/BSE links.
- Key ratios: ROE, ROCE, D/E, ICR, PE, PB, dividend yield.
- 10-year sales/PAT/EPS trend chart.
- Latest cluster label and capital-allocation pattern.
- A **Download Tearsheet PDF** button (see §5).

3.3 Screener

The interactive stock screener. Adjust sliders and drop-downs for min ROE, max D/E, min FCF, sector, min Revenue CAGR, min PAT CAGR, max PE. Results are sorted by composite quality score descending. Six preset buttons apply pre-built filters (see §4).

3.4 Peers

Pick a peer group (IT Services, Private Banks, FMCG, ...) to see every member with percentile ranks across 10 metrics and a radar chart comparing the selected company with the peer-group average and the benchmark (e.g. TCS for IT Services).

3.5 Trends

Multi-year line charts for a selected ticker across sales, operating profit, net profit, EPS, OPM, ROCE, and D/E.

3.6 Sectors

Median ROE / PE / D/E across all 11 broad sectors, plus a drill-down table listing every company in a chosen sector with their KPI values.

3.7 Capital Allocation

Donut chart showing the distribution of the 8-class capital-allocation taxonomy (Reinvestor, Shareholder Returns, Liquidating Assets, Distress Signal, Growth Funded by Debt, Cash Accumulator, Pre-Revenue, Mixed), plus a per-pattern company list.

3.8 Reports

Download centre for generated artifacts: per-company tearsheet PDFs, the 92-page portfolio summary PDF, sector reports, CSV exports (`capital_allocation.csv`, `valuation_flags.csv`, `distress_alerts.csv`) and the OpenAPI / Postman API specs.

4. Screener Presets

Six one-click preset buttons on the Screener screen apply curated filter combinations (defined in spec §25):

Preset	Filter criteria
Quality Compounder	ROE $\geq$ 18%, D/E $\leq$ 0.5, 5yr PAT CAGR $\geq$ 10%, PE $\leq$ 40
Value Pick	ROE $\geq$ 12%, D/E $\leq$ 1.0, PE $\leq$ 15, PB $\leq$ 2
Growth Accelerator	5yr Revenue CAGR $\geq$ 15%, 5yr PAT CAGR $\geq$ 15%, ROE $\geq$ 12%
Dividend Champion	Dividend Yield $\geq$ 2.5%, D/E $\leq$ 0.5, ROE $\geq$ 12%
Debt-Free Blue Chip	D/E = 0, Market Cap = Large Cap, ROE $\geq$ 15%
Turnaround Watch	D/E $\leq$ 2.0, latest PAT $>$ 0, 3yr PAT CAGR flagged TURNAROUND

Presets are additive with the manual sliders; adjust any preset further to refine results.

5. Generating PDF Tearsheets

5.1 Single-company tearsheet

From the Company Profile page, click **Download Tearsheet PDF**. A 2-page PDF is generated on demand and served as a download. You can also fetch a tearsheet directly from the API:

```
curl -o TCS_tearsheet.pdf \  
http://127.0.0.1:8000/api/v1/companies/TCS/tearsheet
```

5.2 Batch generation

To regenerate all 92 tearsheets in one go (takes ~30 seconds), run:

```
python scripts/generate_tearsheets.py
```

Outputs are written to `reports/tearsheets/<TICKER>_tearsheet.pdf`. The 92-page portfolio summary is produced by:

```
python scripts/generate_portfolio_pdf.py
```

5.3 Contents of each tearsheet

- Page 1 - Header with company identity and sector, 10-year KPI table, cluster label, composite quality score, algorithmically generated strengths / weaknesses bullets.
- Page 2 - Sales/PAT trend chart, ROE/ROCE trend, balance-sheet summary, peer radar chart, and capital-allocation pattern callout.

All tables use Paragraph flowables with word-wrap enabled so long text does not overflow cells; the rupee symbol is rendered as "Rs" for font-compatibility.

6. Calling the REST API

The FastAPI server exposes versioned endpoints under `/api/v1/`. Interactive documentation is live at `http://127.0.0.1:8000/docs` (Swagger UI) and `/redoc`. An OpenAPI 3 JSON schema is available at `/export/openapi.json` and a Postman v2.1 collection at `/export/postman.json`.

6.1 Health check

```
curl http://127.0.0.1:8000/api/v1/health
```

6.2 List all companies

```
curl http://127.0.0.1:8000/api/v1/companies/
```

6.3 Get a company profile (TCS)

```
curl http://127.0.0.1:8000/api/v1/companies/TCS
```

6.4 Run the screener (min ROE 18%, sector IT)

```
curl "http://127.0.0.1:8000/api/v1/screener/?min_roe=18&sector=Information+Technology"
```

6.5 Fetch 11 sectors with medians

```
curl http://127.0.0.1:8000/api/v1/sectors/
```

6.6 IT-sector companies

```
curl http://127.0.0.1:8000/api/v1/sectors/Information%20Technology/companies
```

6.7 Peer group & radar

```
curl http://127.0.0.1:8000/api/v1/peers/IT%20Services  
curl http://127.0.0.1:8000/api/v1/companies/TCS/peers/compare
```

6.8 Market-cap history

```
curl http://127.0.0.1:8000/api/v1/market-cap/TCS
```

6.9 Portfolio stats & clusters

```
curl http://127.0.0.1:8000/api/v1/portfolio/stats  
curl http://127.0.0.1:8000/api/v1/portfolio/clusters
```

7. Troubleshooting

Symptom	Likely cause / fix
Dashboard shows 0 companies	Run the ETL (scripts/day1_ingest.py → scripts/populate_ratios.py) to populate db/nifty100.db. Y
API returns 404 for a ticker	Tickers are case-insensitive but must match an NSE symbol exactly (e.g. 'M&M', 'BAJAJ-AUTO
Screener returns fewer than expected	Filters are AND-ed together. Relax one slider or clear the sector filter.
Tearsheet download is slow	Generating a tearsheet involves rendering two charts per company; expect ~200-500 ms. Batch
OPM cross-check warnings in load	The source opm_percentage deviates from $(\text{operating_profit}/\text{sales}) \times 100$ by more than 1 pp; usu
Port 8000 / 8501 already in use	Stop any previous uvicorn/streamlit process or pass a different --port. 'fuser -k 8000/tcp' will fre
PDF shows boxes instead of Rs	The platform uses the ASCII prefix 'Rs' instead of the rupee glyph because Helvetica does not
Cluster count doesn't match spec	Clustering uses random_state=42; if you modified the input data, re-run scripts/day36_clusterin

8. Test Suite & Quality Gates

The project uses pytest with 600+ unit, integration and API tests. To run the suite:

```
pytest tests/ -q                      # full suite
pytest tests/api -q                   # API tests only
pytest tests/perf -v                  # performance tests
pytest tests/ --html=reports/pytest_report.html --self-contained-html
```

Before commit, run Black (line length 100) and Ruff:

```
python -m black src/ tests/ scripts/
python -m ruff check src/ tests/ scripts/
```

Happy analysing! For data issues or feature requests, check the load-audit table in the database and the log files under logs/.